go through the basics of concurrency and threads as well as synchronous and asynchronous ways of program execution before diving into `asyncio`.

`Event loops` are the central entity on which the entire `asyncio` module functions. An event loop registers a task for execution and the task is executed, delayed or cancelled based on our requirements. It is the job of event loop to manage I/O for all the registered tasks.

A `coroutine` pretty much resembles a function but it is capable of returning the control to the caller function without losing the state of execution, the advantage being they take very less memory to execute. Coroutines in Python are prefixed with a async keyword. It should be noted that when calling a coroutine, it will not actually execute rather return a coroutine object which is passed to the event loop for execution.

We will look at an example hereinafter to strengthen our concepts of asyncio, coroutines and event loops. The end goal of this program is to calculate factorial of a given number. If multiple numbers are provided to the program as input, it should calculate factorials of smaller numbers first and should delay the calculation of larger numbers since they will take a longer time to compute. If we try to do the aforementioned calculation synchronously, big numbers will block the calculation of smaller numbers, which is not desired. Hence we bring in an asynchronous approach.

We write a subroutine for factorial calculation, which sleeps for 0.01 seconds after multiplication of every five numbers. When the subroutine calls `await` and sleeps, it passes the control back to the event loop which triggers the next submitted task. This ensures that all tasks get an execution window in a round-robin fashion and none of the tasks enter starvation. Hence, the smallest number finishes first and largest number finishes last. This is how an asyncio module handles asynchronous tasks quite gracefully with minimal overhead.

```
import asyncio

async def fact(number):
    result = 1
      print("Start Factorial Calculation for: {}".
format(number))
    for i in range(number):
        result = result * (i+1)
        if i % 5 == 0:
            await asyncio.sleep(0.01)
```